

Phụ lục. Giải thích MVP

MVP là gì?

MVP là viết tắt của **Minimum Viable Product**, nghĩa là **phiên bản sản phẩm khả dụng tối thiểu**. Trong bối cảnh bài tập lớn, MVP là phiên bản nhỏ nhất của phần mềm nhưng vẫn phải:

- chạy được;
- thực hiện đúng nghiệp vụ chính;
- có cấu trúc mã đủ rõ để thể hiện design pattern, SOLID và unit test.

Nói cách khác, nhóm không cần làm đủ mọi thứ, mà cần làm đúng phần quan trọng nhất.

Vì sao phải làm theo MVP?

- tránh ôm quá nhiều chức năng trong thời gian có hạn;
- tập trung vào chất lượng kiến trúc và kiểm thử;
- có thời gian hoàn thiện phần nghiệp vụ cốt lõi;
- giảm rủi ro làm dở nhiều tính năng nhưng không hoàn thành được sản phẩm.

MVP không có nghĩa là làm qua loa

MVP không có nghĩa là:

- giao diện sơ sài đến mức không dùng được;
- code tạm bợ, không tách lớp;
- bỏ qua unit test;
- nhồi nhiều nút bấm nhưng nghiệp vụ chính chưa chạy ổn.

MVP đúng nghĩa là:

- ít chức năng hơn;
- nhưng mỗi chức năng cốt lõi phải chạy ổn, có tổ chức mã rõ ràng và có kiểm thử.

Cách xác định MVP cho đề tài

Khi chọn phạm vi MVP, nhóm nên tự hỏi 4 câu:

1. Người dùng chính của ứng dụng là ai?
2. Nghiệp vụ quan trọng nhất mà ứng dụng phải làm được là gì?
3. Những chức năng nào bắt buộc phải có để nghiệp vụ đó chạy trơn vẹn?
4. Những chức năng nào có thể để lại cho phần mở rộng nếu còn thời gian?

Nhóm chỉ nên giữ lại các chức năng trả lời trực tiếp cho câu 2 và 3.

Ví dụ minh họa

Đề tài: Ứng dụng tạo đề trắc nghiệm

MVP nên có:

- thêm, sửa, xóa câu hỏi;
- phân loại câu hỏi theo chủ đề hoặc độ khó;
- sinh 1 đề kiểm tra theo số câu yêu cầu;
- xem trước đề đã sinh;
- lưu đề đã sinh.

Chưa bắt buộc trong MVP:

- đăng nhập tài khoản;
- phân quyền nhiều vai trò;
- xuất PDF đẹp;
- đồng bộ cloud;
- thống kê phức tạp;
- giao diện quá cầu kỳ.

Như vậy, MVP vẫn đủ để chứng minh ứng dụng giải quyết được bài toán chính, nhóm biết tổ chức dữ liệu và service, đồng thời có áp dụng design pattern và viết unit test.

Nguyên tắc chốt phạm vi MVP

- Ưu tiên 1 luồng nghiệp vụ hoàn chỉnh hơn là nhiều chức năng rời rạc.
- Ưu tiên chạy ổn định hơn là nhiều tính năng phụ.
- Ưu tiên kiến trúc tốt + test được hơn là giao diện quá phức tạp.
- Chỉ làm tính năng mở rộng sau khi phần MVP đã hoàn chỉnh.

Gợi ý kiểm tra nhanh trước khi chốt MVP

Nhóm có thể dùng checklist sau:

- Ứng dụng đã có một mục tiêu chính rất rõ chưa?
- Toàn bộ chức năng hiện tại có phục vụ trực tiếp mục tiêu đó không?
- Có chức năng nào "hay nhưng chưa cần" có thể cắt bỏ không?
- Nhóm đã đủ thời gian để viết unit test cho phần lỗi chưa?
- Nếu chỉ demo 5 phút, ứng dụng có thể chứng minh được giá trị chính không?

Nếu câu trả lời là có, phạm vi MVP thường đã phù hợp.